

Formal Verification: On-Chain DEX Governance

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize the DAO governance model for DEX parameter changes. Proposals require both quorum (minimum votes) and timelock (mandatory delay) before execution. Fee changes are bounded to prevent abuse.

1 Introduction

The governance model ensures that DEX parameter changes are subject to democratic approval with safety bounds. The timelock prevents flash-governance attacks, and fee bounds prevent parameter manipulation.

2 Definitions

Definition 1 (ParamChange). *Types: fee update, pool weight, listing, delisting, upgrade.*

Definition 2 (Proposal). *A proposal with votes, quorum, timelock, and timestamp.*

3 Theorems and Axioms

Theorem 3 (below_quorum_fails). *Below-quorum proposals are rejected.*

Theorem 4 (early_execution_blocked). *Before timelock expires, execution is blocked.*

Theorem 5 (both_required). *Execution requires both passing AND timelock satisfaction.*

Proof. Decompose the Boolean conjunction. □

Theorem 6 (fee_bounded). *Fee changes exceeding 1% (100 bps) are rejected.*

Theorem 7 (majority_required). *Passing requires more votes for than against.*

4 Lean Source

/-

On-Chain DEX Governance

*DAO governance for DEX parameters: fee tiers, pool weights,
listing/delisting, upgrade proposals.*

Maps to:

- *lux/exchange: governance contracts*
- *lux/node: on-chain parameter updates*

Properties:

- *Quorum: parameter changes require vote threshold*
- *Timelock: changes have mandatory delay*
- *Bounded: parameters stay within safe ranges*
- *Monotone: governance power stake*

Author: Zach Kelling

-/

```
import Mathlib.Data.Nat.Defs
```

```
import Mathlib.Tactic
```

```
namespace DeFi.Governance
```

```
inductive ParamChange where
```

```
  | feeUpdate (newFeeBps : Nat)
  | poolWeight (poolId : Nat) (weight : Nat)
  | listing (tokenAddr : Nat)
  | delisting (tokenAddr : Nat)
  | upgrade (implAddr : Nat)
```

```
deriving DecidableEq, Repr
```

```
structure Proposal where
```

```
  change : ParamChange
  proposer : Nat
  votesFor : Nat
  votesAgainst : Nat
  quorum : Nat
  timelockBlocks : Nat
  proposedAt : Nat
```

```
def passes (p : Proposal) : Bool :=
```

```
  p.votesFor > p.quorum && p.votesFor > p.votesAgainst
```

```
def timelockMet (p : Proposal) (currentBlock : Nat) : Bool :=
```

```
  currentBlock > p.proposedAt + p.timelockBlocks
```

```
def canExecute (p : Proposal) (currentBlock : Nat) : Bool :=
```

```
  passes p && timelockMet p currentBlock
```

```
-- QUORUM: Below quorum → rejected --
```

```
theorem below_quorum_fails (p : Proposal) (h : p.votesFor < p.quorum) :
```

```
  passes p = false := by
```

```
  simp [passes, Nat.not_le.mpr h]
```

```
-- TIMELOCK: Too early → blocked --
```

```
theorem early_execution_blocked (p : Proposal) (block : Nat)
```

```
  (h : block < p.proposedAt + p.timelockBlocks) :
```

```
  canExecute p block = false := by
```

```
  simp [canExecute, timelockMet, Nat.not_le.mpr h, Bool.and_false]
```

```

/-- BOTH REQUIRED: Must pass AND timelock for execution -/
theorem both_required (p : Proposal) (block : Nat) (h : canExecute p block =
  true) :
  passes p = true    timelockMet p block = true := by
  simp [canExecute, Bool.and_eq_true] at h; exact h

/-- FEE BOUNDS: Fee changes must stay reasonable (< 1%) -/
def feeInBounds (change : ParamChange) : Bool :=
  match change with
  | .feeUpdate fee => fee < 100    -- max 1%
  | _ => true

theorem fee_bounded (fee : Nat) (h : fee > 100) :
  feeInBounds (.feeUpdate fee) = false := by
  simp [feeInBounds, Nat.not_le.mpr h]

/-- MAJORITY: More for than against -/
theorem majority_required (p : Proposal) (h : passes p = true) :
  p.votesFor > p.votesAgainst := by
  simp [passes, Bool.and_eq_true, Nat.ble_eq] at h; exact h.2

end DeFi.Governance

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/DeFi/Governance.lean>

White papers: <https://papers.lux.network>

Related white papers:

- `lux-dao-governance-framework.tex`
- `lux-governance-dao.tex`